第 10 章練習解答　大量儲存結構

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS10-E01　磁碟排程

1. FCFS：50→82→170→43→140→24→16→190。 總移動 32+88+127+97+116+8+174=642。
2. SSTF：50→43→24→16→82→140→170→190；每一步選絕對距離最短者。 總移動 7+19+8+66+58+30+20=208。
3. 連續出現靠近磁頭的請求時，遠端請求可能長期不變近，因此可能 starvation。

檢查。 列出請求序列與移動距離以示範，不只給名稱。

OS10-E02　RAID 5 與 RAID 1

1. RAID 5 有 3 份資料與 1 份 parity，容量約 3 個 disk，任意一顆故障可重建。
2. 題設兩組雙碟 mirror 合計容量為 2 個 disk；每組可容忍該組一碟失效。兩組各失效一碟仍可存取兩份邏輯資料，同組兩碟全失效則該組資料遺失。單一四副本 RAID 1 的容量則只有 1 個 disk，不能混稱。
3. RAID 1 通常降低單碟重建時需要讀取多顆剩餘碟的壓力，但容量成本更高。實際可靠度還要看故障域、rebuild 速率與 metadata。

檢查。 容量與故障假設要分開回答。

OS10-E03　穩定寫入

1. 成功可能只代表資料被作業系統接受，尚未到持久媒介。檔案 API 若未包含同步承諾，就不保證斷電後存在。
2. 需明確 flush/barrier 或交易提交，使資料及必要 metadata 到達裝置持久媒介，並處理裝置 cache。
3. mirror 可增加指定硬體故障下的副本，但如果兩個副本都在 volatile cache 或系統回報過早，仍不能保證資料到達 stable storage。

檢查。 把 cache、device acknowledgment 與 durable commit 三階段分開。

第 10 章練習解答　大量儲存結構 （續）

OS10-E04　RPM、IOPS與response分母

1. 一圈60000/7200=25/3ms；平均rotation25/6ms；transfer=(64KiB/128MiB)×1000=125/256ms。Service=5+25/6+125/256=7415/768≈9.65495ms。
2. response=15+7415/768≈24.65495ms；零queue且一請求服務完才下一個時IOPS=1000/(7415/768)≈103.574。不把queue wait算入media service，也不能把此倒數套多queue重疊。
3. 4KiB×1000=3.90625MiB/s；64KiB×1000=62.5MiB/s。相同IOPS不同bytes/request便不同吞吐，順序1GiB的純transfer下界另為8s。

檢查。 以相同units、故障模型、逐步狀態與完成界線重算。

OS10-E05　Flash GC的mapping與WA

1. 先新寫P4並令L1→P4、P1 invalid；copy L0:P0→P5、L2:P2→P6、L3:P3→P7，更新安全後erase B0。最後L0→5、L1→4、L2→6、L3→7。
2. Host1page；無discard new1+GC3=4physical programs，WA4。L2不再live時GC只copy2，programs3、WA3。排除mapping/其他controller開銷與erase成本，分母皆host page programs。
3. Discard說logical資料不再需要，不保證立即抹除或安全擦除。Crash時還要讓新mapping/data可恢復再erase含唯一live資料的old block；正常trace的WA計數不自動證FTL crash一致性。

檢查。 以相同units、故障模型、逐步狀態與完成界線重算。

OS10-E06　LOOK、SCAN與回繞計費

1. LOOK53→37→14→65→67→98→122→124→183，39+169=208。SCAN再經0才上行，53→37→14→0→65→67→98→122→124→183，53+183=236。208不能叫本題SCAN。
2. 不是。SSTF53→65→67→37→14→98→122→124→183，距離12+2+30+23+84+24+2+59=236；此輸入總量恰同，規則/順序不同。LOOK208已證SSTF非全域最小。
3. C-SCAN經199→0再14→37，146+199+37=382；C-LOOK自183跳14再37，130+169+23=322，皆含回繞。SSTF無限starvation須新requests持續在附近出現；固定有限batch不能據此宣稱真的永遠未完成。

檢查。 以相同units、故障模型、逐步狀態與完成界線重算。

第 10 章練習解答　大量儲存結構 （續）

OS10-E07　Parity、write hole與第二關係

1. P=3C xorA5 xor5A=C3；D1=C3 xor3C xor5A=A5。D1改A4時Pnew=C3 xorA5 xorA4=C2。Cold small-write需讀old data/parity再寫new data/parity，共2R+2W，不計log/cache。
2. 重建D2=C3 xor3C xorA4=5B，正確原值5A。Dirty parity已破壞一致性前提；一disk fault tolerance不是任意write-hole再加disk loss的保證。
3. Q=3C xor57 xor75=1E；扣D0後Pmissing=FF,Qmissing=22。(2+4)B=22 xor(2・FF)=C1，inverse6=7A，B=7A・C1=5A，A=FF xor5A=A5。乘法須用11D field，不做整數乘法。
4. A+B=Pmissing與αA+βB=Qmissing，在α≠β時α+β非零可逆，所以唯一B再A。若Q只是P副本，方程相依，兩unknown不可唯一解。

檢查。 以相同units、故障模型、逐步狀態與完成界線重算。

OS10-E08　雙副本穩定更新的crash prefixes

1. C1途中可能v1/v2/torn，但C2保留valid v1，至少可恢復v1；若C1完整valid v2可選v2。C2途中C1已durable v2，因此必能回v2，不是舊v1。
2. v1 body=big-endian gen1+ASCII V1加502zero，CRC BF4CA2E8；v2同法CRC4A112E80。CRC覆蓋前508bytes，成功後按generation選最高，不按CRC大小。CRC只對本模型注入tears檢測，不是無碰撞或authentication。
3. 寫一份時另一份保持valid；C1 new flush後才改C2，所以至少一good copy歸納維持。Recovery保留選中的good，先修另一份並flush，修復中再crash也保留good。兩份同時寫/flush不可信/共同lost破壞前提。
4. 未ACK operation可恢復old或完整new，不能把未回覆等同沒做。CPU memory barrier不等device flush；排序不等atomic no-tear，FUA/flush依device contract。此單record協議不自動保證multi-record transaction。

檢查。 以相同units、故障模型、逐步狀態與完成界線重算。

第 10 章練習解答　大量儲存結構 （續）

OS10-E09　拓樸、block/file介面與swap引用

1. A+C：0+1兩stripe各缺一，不可用；1+0每pair仍B/D可用。A+B：0+1尚有完整CD stripe；1+0的AB pair全失。拓樸不同，不能只說都有半數冗餘。
2. NAS file server解析file handle/offset到block；SAN host本機filesystem轉LBA再送block target。共用LUN本身未提供filesystem lock/coherence，兩host獨占型filesystem並寫可破壞metadata，需協調設計。
3. 不可，P2仍應讀41。模型swap-PTE refs2→1→P2讀回後0，還須無in-flight I/O/cache保留ownership才可回收。P1新版本必要時另配backing；此計數不當Linux swap_map實作。
4. A LBA2048、count204800、exclusive end206848；B start206848、count409600、exclusive end616448。不重疊；此是教學partition區間非完整GPT規格。

檢查。 以相同units、故障模型、逐步狀態與完成界線重算。

原創驗算程式附錄: storage_models.py

```
"""Offline/serial storage teaching models. No real device I/O is performed."""
from fractions import Fraction
from functools import reduce
from operator import xor
import zlib


def movement(path):
    return sum(abs(right-left) for left,right in zip(path,path[1:]))


def schedule(head,requests,policy,maximum=199):
    if not 0<=head<=maximum or any(not 0<=item<=maximum for item in requests):
        raise ValueError("cylinder outside device")
    pending=list(requests)
    if policy=='FCFS':return [head]+pending
    if policy=='SSTF':
        path=[head]
        while pending:
            index=min(range(len(pending)),key=lambda i:(abs(pending[i]-
path[-1]),pending[i],i))
            path.append(pending.pop(index))
        return path
    prefix=[head]+[item for item in pending if item==head]
    lower=sorted(item for item in pending if item<head)
    upper=sorted(item for item in pending if item>head)
    if policy=='LOOK-down':return prefix+lower[::-1]+upper
    if policy=='SCAN-down':return prefix+lower[::-1]+([0] if upper else [])+upper
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 10 章練習解答　大量儲存結構 (續)

```
    if policy=='C-LOOK-up':return prefix+upper+lower
    if policy=='C-SCAN-up':return prefix+upper+([maximum,0] if lower else [])+lower
    raise ValueError("unknown policy")


def service_ms():
    rotation=Fraction(60000,7200)/2
    transfer=Fraction(64*1024*1000,128*1024*1024)
    return Fraction(5)+rotation+transfer


def parity(values):
    return reduce(xor,values,0)


def gf_multiply(a,b):
    result=0
    while b:
        if b&1:result^=a
        a<<=1
        if a&0x100:a^=0x11D
        b>>=1
    return result


def gf_inverse(value):
    if not value:raise ValueError("zero has no inverse")
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 10 章練習解答　大量儲存結構 （續）

```
    result=1
    for _ in range(254):result=gf_multiply(result,value)
    return result


def recover_two(p_missing,q_missing,alpha,beta):
    if alpha==beta:raise ValueError("dependent parity equations")
    b=gf_multiply(gf_inverse(alpha^beta),q_missing^gf_multiply(alpha,p_missing))
    return p_missing^b,b


def record(generation):
    payload=f"V{generation}".encode()
    payload+=bytes(504-len(payload))
    body=generation.to_bytes(4,'big')+payload
    return body+zlib.crc32(body).to_bytes(4,'big')


def valid(data):
    return len(data)==512 and zlib.crc32(data[:508])==int.from_bytes(data[508:],'big')


def recover(copies):
    good=[item for item in copies if valid(item)]
    if not good:raise ValueError("no valid copy remains")
    return max(good,key=lambda item:int.from_bytes(item[:4],'big'))


def flash_gc(trim_l2=False):
    mapping={0:0,1:1,2:2,3:3};trace=[dict(mapping)]
    mapping[1]=4;physical_programs=1;trace.append(dict(mapping))
    if trim_l2:del mapping[2]
    target=5
    for logical,physical in list(mapping.items()):
        if physical<4:
            mapping[logical]=target;target+=1;physical_programs+=1
    trace.append(dict(mapping))
    return {'mapping_trace':trace,'erased_pages':[0,1,2,3],
            'host_page_writes':1,'physical_page_programs':physical_programs}


if __name__=='__main__':
    requests=[98,183,37,122,14,124,65,67]
    for policy in ['FCFS','SSTF','LOOK-down','SCAN-down','C-SCAN-up','C-LOOK-up']:
        path=schedule(53,requests,policy)
        print(policy,path,movement(path))
    print('service_ms:',service_ms(),float(service_ms()))
    print('parity:',hex(parity([0x3C,0xA5,0x5A])))
    print('FTL:',flash_gc())
    for generation in (1,2):print('record',generation,record(generation)[508:].hex())
    print('recovered generation:',int.from_bytes(recover([record(2),record(1)])[:4],'big'))
```

第 10 章練習解答　大量儲存結構 （續）

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study